DATA/CS 172: Python for Data Analysis

N-D arrays and images

Prof. Travis Mandel

11/4/2024

UNIVERSITY of HAWAI'I®
HILO

Announcements

- Stay on top of labs
- Should be working on Program 2

Today

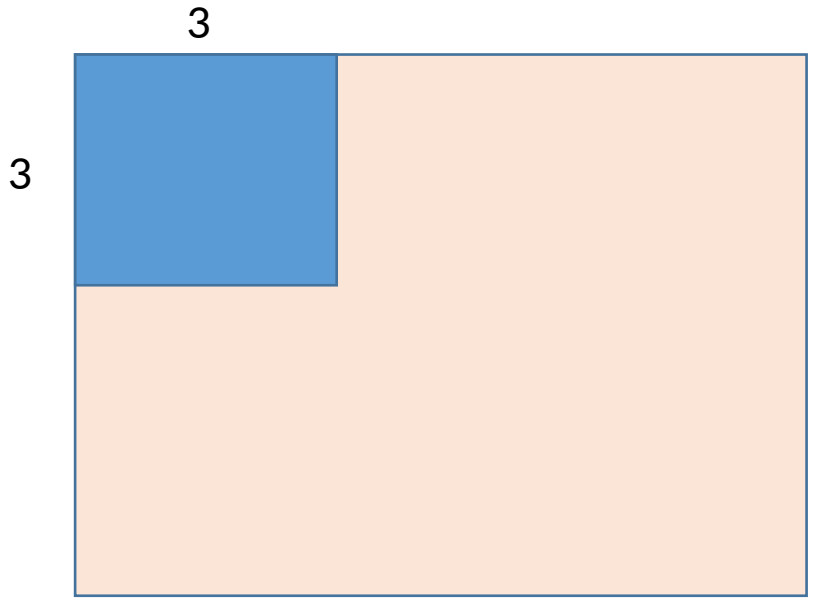
- Multi-dimensional numpy arrays

Multi-d numpy, so far

- Arr.shape
- Indexing with arr[firstIndex, secondIndex]

Slicing

- You can still slice on multi-dimensional arrays!
  Just get to pick which dimension you slice on!



- Arr2d[:,1] gets everything in the second column
- Arr2d[:3,:3] gets the upper left corner 3x3
- Arr2d[:] is shorthand for arr2d[:,:]
  - Good style to put in : for the dimensions you want everything on

Operations

- As with 1D arrays, operations apply to all elements
  - arr2d *2 doubles all elements
- As with 1D arrays, operations across arrays work element-wise
  - Assuming they are the same size, arr1 * arr2 does arr1[0,0] * arr2[0,0], arr1[0,1] * arr2[0,1], etc.
- As with 1D arrays, can apply operations to a slice and have it apply to all elements of that slice

Boolean filtering/masking/indexing

- Sometimes we want to apply an operation only to certain elements
  - Slices can help us apply to certain indexes, but not to certain values…
- Can create a Boolean filter/mask
- dogfilter = (arr == ‘dog’)
  - Array of booleans
- arr[dogfilter] = ‘cat’ ← changes all dogs to cats
- Can use <. <=, etc
- Want to combine with logic?
  - Careful – and & or don’t work
  - Need to use bitwise symbols
    - & = and
    - | = or
    - ~ = not
  - Example: ((arr > 6) & (arr < 10))
    - Note: need parentheses!

Reshaping

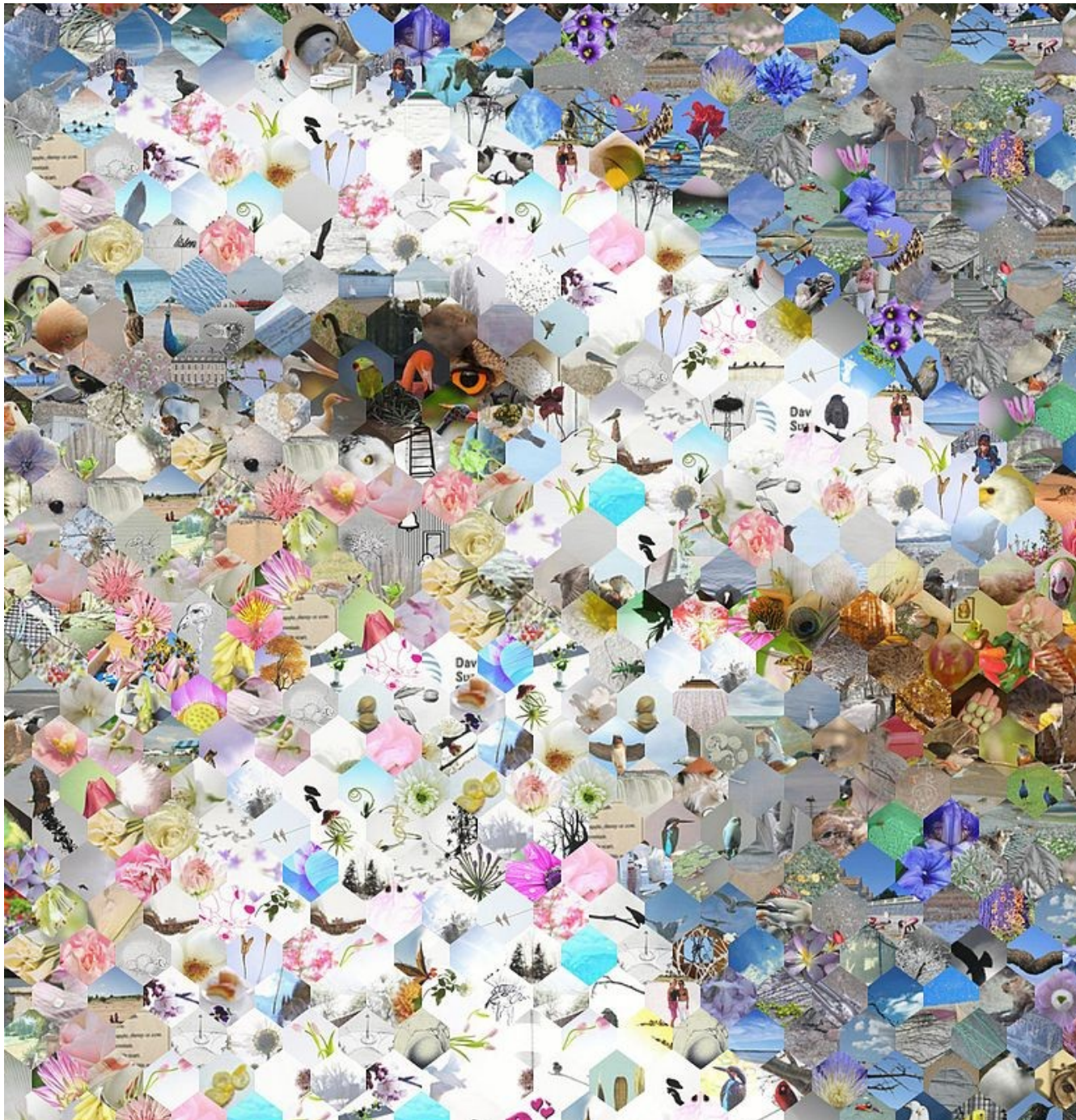
- Frequently a library will take one shape (e.g. 1D array) when you have another shape (e.g. 3D array)

- arr.reshape((newRows,newCols,…))
  - Defaults to row-major order: walks through first row, then second row, etc.

- Does not make a copy
- Shorthand for flattening down to 1D is called arr.ravel()
  - arr.flatten() also exists but it makes a copy (want to avoid)

Images

- So far we have talked about text data, CSV data, JSON data
  - What about image data?
- Image data is large and has multiple dimensions
- Therefore, in Python, we almost always work with images as multi-dimensional numpy arrays!



Loading images

- Wait… why would we model an image as a 3D array?
- **To read:**

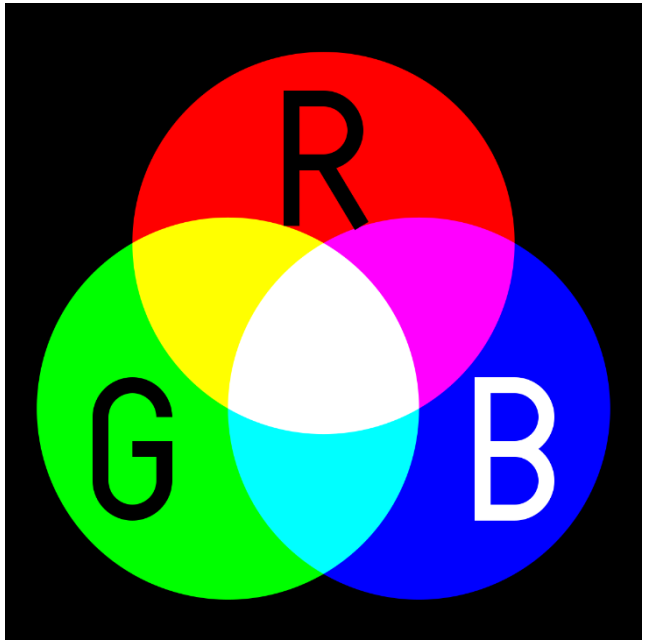
```
import matplotlib.image as mpimg
img = mpimg.imread("cat2.png")
```

- **To show:**

```
import matplotlib.pyplot as plt
plt.imshow(img)
plt.show()
```

What is the shape?

- Why 3d?
  - Red – either between 0-255 (as int) or 0-1 (as float)
  - Green – either between 0-255 or 0-1
  - Blue – either between 0-255 or 0-1
  - Should be in this order
- Darken an image – multiply by a small value
- Need to make sure it doesn't go out of range – can be a problem with lightening



Lab out

- Practice using numpy arrays to manipulate image!